



6.9 ResponseEntity Object

- ResponseEntity is a **generic type**
- Used to **Manipulate** the HTTP Response
- Represents the whole HTTP response: **status code, headers, and body**
- provides two nested builder interfaces: **HeadersBuilder** and its subinterface, **BodyBuilder**
- **Alternate** for ResponseEntity - @ResponseBody, @ResponseStatus and HttpServletResponse

```
@GetMapping("/hello")
```

```
    ResponseEntity<String> hello() {
```

```
        return new ResponseEntity<>("Hello World!", HttpStatus.OK);
```

```
    }
```

6.9 @RequestBody annotation

- If a method parameter is annotated with @RequestBody, Spring will bind the incoming HTTP request body(for the URL mentioned in @RequestMapping for that method) to that parameter.
- While doing that, Spring will use HTTP Message converters to convert the HTTP request body into domain object [**deserialize request body to domain object**], based on **Accept** header present in request.

```
@RestController
public class EmployeeController {
    @Autowired
    IEmployeeService empService;
    @RequestMapping(value = "/employee/create/", consumes = MediaType.APPLICATION_JSON_VALUE,
headers="Accept=application/json",method = RequestMethod.POST)
    public List<Employee> createEmployee(@RequestBody Employee emp) {

        empService.addEmployee(emp);
        return empService.getAllEmployee();
    } }
```



6.10 Cross-Origin Resource Sharing (CORS)

- **CORS** (Cross-origin resource sharing) allows a webpage to request additional resources into browser from **other domains** e.g. fonts, CSS or static images from CDNs.
- Helps in serving web content from multiple domains into browsers who usually have the same-origin security policy.
- **Spring CORS** support in Spring MVC application at method level and global level.
- **@CrossOrigin** allows all origins, all headers, the HTTP methods specified in the **@RequestMapping** annotation and a **maxAge of 30 minutes**.

6.10 @CrossOrigin Annotation Attributes



Origins - List of allowed origins. It's value is placed in the Access-Control-Allow-Origin header of both the pre-flight response and the actual response.

- * - means that all origins are allowed.
- If undefined, all origins are allowed.

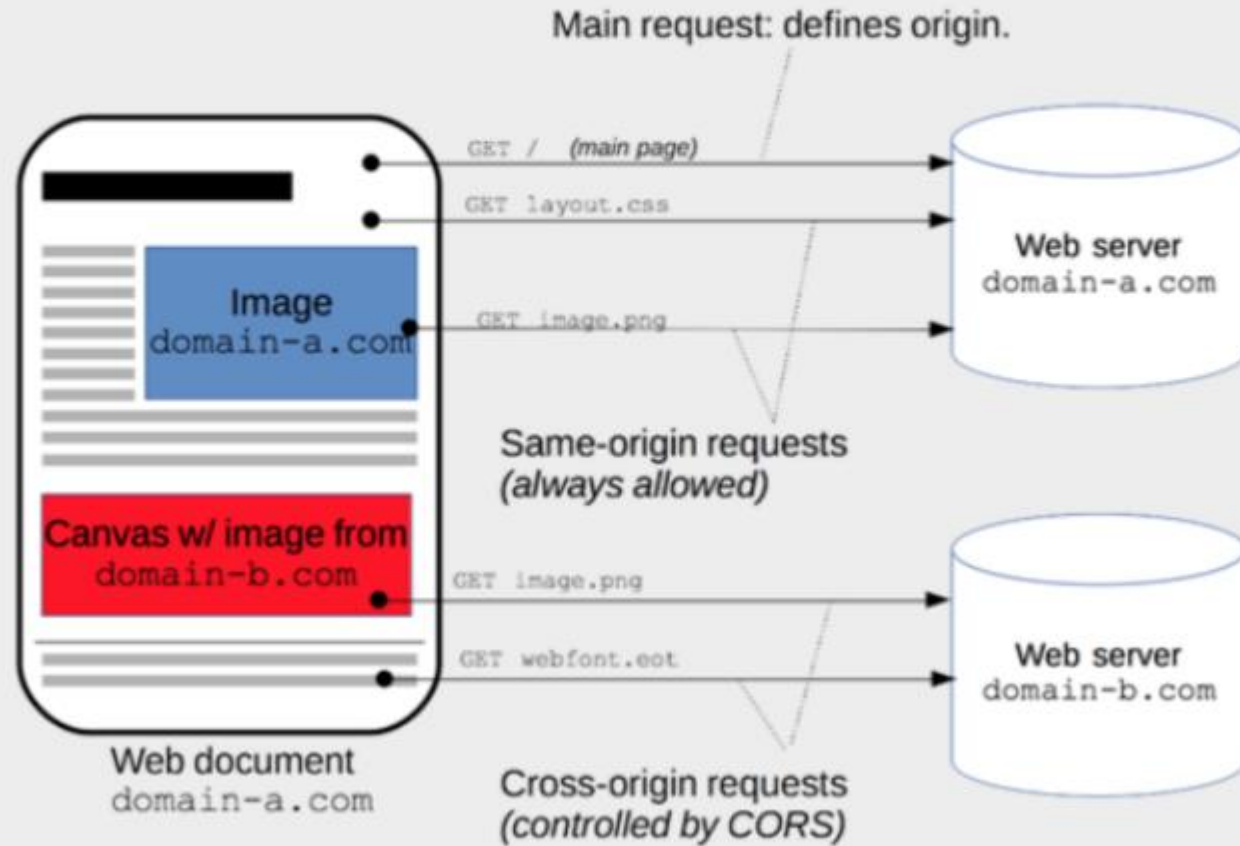
allowedHeaders - List of request headers that can be used during the actual request. Value is used in preflight's response header Access-Control-Allow-Headers.

- * - means that all headers requested by the client are allowed.
- If undefined, all requested headers are allowed.

methods - List of supported HTTP request methods. If undefined, methods defined by RequestMapping annotation are used.

exposedHeaders - List of response headers that the browser will allow the client to access. Value is set in actual response header Access-Control-Expose-Headers.

- If undefined, an empty exposed header list is used.



Reference : <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

The CORS mechanism supports secure cross-domain requests and data transfers between browsers and web servers. Modern browsers use CORS in an API container such as [XMLHttpRequest](#) or [Fetch](#) to help mitigate the risks of cross-origin HTTP requests



6.10 @CrossOrigin Annotation Attributes

- **allowCredentials** - It determine whether browser should include any cookies associated with the request.
 - false - cookies should not included.
 - "" (empty string) - means undefined.
 - true - pre-flight response will include the header Access-Control-Allow-Credentials with value set to true.
 - If undefined, credentials are allowed.
- **maxAge** - maximum age (in seconds) of the cache duration for pre-flight responses. Value is set in header Access-Control-Max-Age.
 - If undefined, max age is set to 1800 seconds (30 minutes).

Cross-Origin Resource Sharing (CORS)

```
@CrossOrigin(origins = "http://localhost:4200")
@RestController
public class CountryController {
    @Autowired
    private ICountryService service;

    // @CrossOrigin(origins = "http://localhost:4200")
    @RequestMapping(value = "/countries/search/{id}", method =
        RequestMethod.GET, headers = "Accept=application/json")
    public Country getCountry(@PathVariable int id) {
        return service.searchCountry(id);
    }
}
```



7.1 Exception handling at controller level

we will define a method to handle exceptions, and annotate that with `@ExceptionHandler`

```
@RestController
public class CountryController{

    @ResponseStatus(value=HttpStatus.NOT_FOUND,
reason="Country with this id not present")
    @ExceptionHandler({Exception.class})
    public void handleException() {

    }
}
```

`@ExceptionHandler` along with `@ResponseStatus` to map the exception to the custom method in controller which can handle all exception in that controller. In `@ExceptionHandler` annotation we can include the Exception classes which we need to handle for this controller

7.2 Exception handling at application level

Spring 3.2 brings support for a global `@ExceptionHandler` with the new `@ControllerAdvice` annotation.

```
@ControllerAdvice
public class DemoExceptionHandler {
    @ResponseBody
    @ResponseStatus(value=HttpStatus.NOT_FOUND)
    @ExceptionHandler(value = {Exception.class})
    protected ErrorInfo handleConflict(Exception ex,
    HttpServletRequest req) {
        String bodyOfResponse = ex.getMessage();// "Country
with this id not present";
        String uri = req.getRequestURL().toString();
        return new ErrorInfo(uri,bodyOfResponse);
    }
}
```

The new annotation allows the multiple scattered `@ExceptionHandler` from before to be consolidated into a **single, global error handling component**. The actual mechanism is extremely simple but also very flexible: it allows full control over the body of the response as well as the status code it allows mapping of several exceptions to the same method, to be handled together it makes good use of the newer RESTful *ResponseEntity* response

```
public class ErrorInfo {
    private String url;
    private String message;
    public ErrorInfo(String url, String message) {
        this.url = url;
        this.message = message;
    }
    public String getUrl() {
        return url;
    }
    public void setUrl(String url) {
        this.url = url;
    }
    public String getMessage() {
        return message;
    }
    public void setMessage(String message) {
        this.message = message;
    }
}
```

@ExceptionHandler

- @ExceptionHandler annotation
 - It handles exceptions by creating a method to catch specific exceptions and return a customized response, often within a @ControllerAdvice class for global handling.
- @ControllerAdvice
 - can define an ExceptionHandler method for a custom MyCustomException that returns a tailored ErrorResponse object to the user, rather than a generic error.

Two ways to use @ExceptionHandler

- Controller-level: When placed directly within a @Controller or @RestController class, the @ExceptionHandler method will only handle exceptions for that specific controller.
- This is useful for handling exceptions specific to a particular set of endpoints.
- Global-level: By placing the @ExceptionHandler method inside a class annotated with @ControllerAdvice, you can centralize exception handling for all controllers in your application.
- This is the recommended approach for providing a consistent error-handling strategy across your entire application.

@ExceptionHandler example with @ControllerAdvice

- The most common and effective pattern is to combine @ExceptionHandler with @ControllerAdvice for global error handling.

1. Define custom exceptions

Create a custom exception for a specific business rule, such as when a requested resource is not found.

java

```
public class ResourceNotFoundException extends RuntimeException {
```

```
    private static final long serialVersionUID = 1L;
```

```
    public ResourceNotFoundException(String msg) {
```

```
        super(msg);
```

```
    }
```

```
}
```

ErrorMessage.java

```
public class ErrorMessage {  
    private int statusCode;  
    private String message;  
  
    public ErrorMessage(int statusCode, String message {  
        this.statusCode = statusCode;  
        this.message = message;  
    }  
    // getter, setter
```


global exception handler

2. Create a global exception handler

Create a class with the `@ControllerAdvice` annotation. This class will contain `@ExceptionHandler` methods to catch specific exceptions thrown from any controller.

The methods can then build a custom, user-friendly `ResponseEntity`.

Throw the exception from a controller

@ControllerAdvice

```
public class ControllerExceptionHandler {
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
```

```
    public ResponseEntity<ErrorMessage> resourceNotFoundException(ResourceNotFoundException ex) {
```

```
        ErrorMessage message = new ErrorMessage(
```

```
            HttpStatus.NOT_FOUND.value(),
```

```
            ex.getMessage());
```

```
        return new ResponseEntity<ErrorMessage>(message, HttpStatus.NOT_FOUND);
```

```
    }
```

```
    @ExceptionHandler(Exception.class)
```

```
    public ResponseEntity<ErrorMessage> globalExceptionHandler(Exception ex) {
```

```
        ErrorMessage message = new ErrorMessage(
```

```
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
```

```
            ex.getMessage());
```

```
        return new ResponseEntity<ErrorMessage>(message, HttpStatus.INTERNAL_SERVER_ERROR);
```

```
    }
```

```
}
```

In controller no need to handle using try and catch.

```
@RestController

@RequestMapping("/")

public class CustomerController {

    @Autowired

    private CustomerRepository customerRepository;


    @GetMapping("/customer")

    public ResponseEntity<List<Customer>> getAll() {

        if (customerRepository.findAll().isEmpty()) {

//            return new ResponseEntity<>(HttpStatus.NO_CONTENT);

            throw new ResourceNotFoundException("list is empty");

        } else

            return new ResponseEntity<>(customerRepository.findAll(),

HttpStatus.OK);

    }

    @PostMapping("/customer")

    public ResponseEntity<Customer> addCustomer(@RequestBody

Customer customer) {

        return new

ResponseEntity<Customer>(customerRepository.save(customer),

HttpStatus.FOUND);

    }

}
```

```
@PutMapping("/customer")

    public ResponseEntity<Customer> updateCustomer(@RequestBody Customer

customer) {

        Customer c = customerRepository.findById(customer.getId()).orElseThrow(() ->

new ResourceNotFoundException("no customer found with id "

+customer.getId()));

        return new ResponseEntity<>(customerRepository.save(customer),

HttpStatus.FOUND);

    }

    @GetMapping("/customer/{id}")

    public Customer searchCustomerById(@PathVariable Integer id){

        public ResponseEntity<Customer> searchCustomerById(@PathVariable Integer

id) {

            Customer c = customerRepository.findById(id).orElseThrow(() -> new

ResourceNotFoundException("no customer found with id " + id));

            return new ResponseEntity<>(c, HttpStatus.OK);

        }

        @DeleteMapping("/customer/{id}")

        public ResponseEntity<Customer> deleteCustomerById(@PathVariable("id") int

id) {

            Customer c = customerRepository.findById(id).orElseThrow(() -> new

ResourceNotFoundException("no customer found with id " + id));

            customerRepository.deleteById(id);

            return new ResponseEntity<>(c, HttpStatus.OK);

        }

    }

}
```